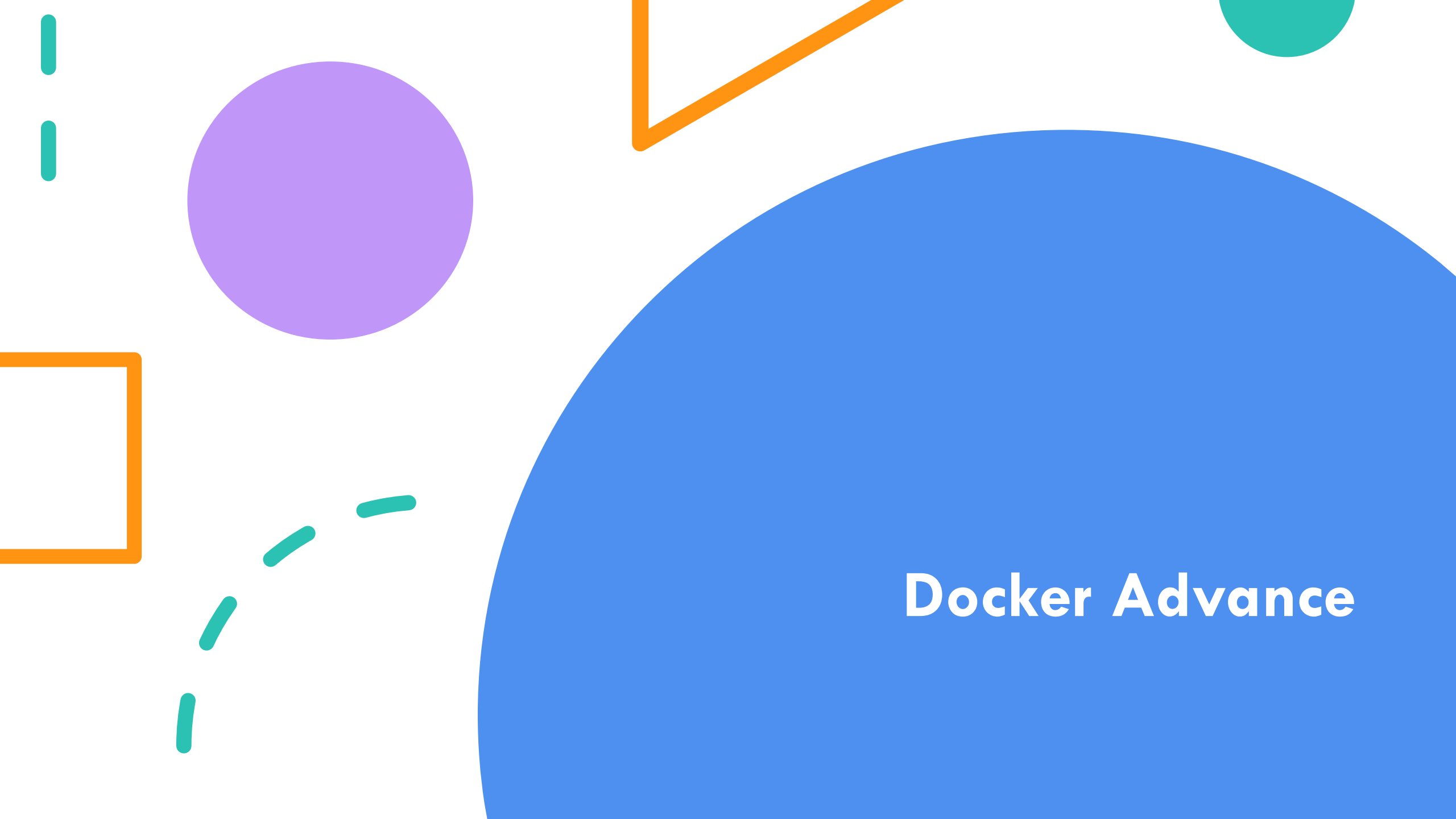




Docker Advance

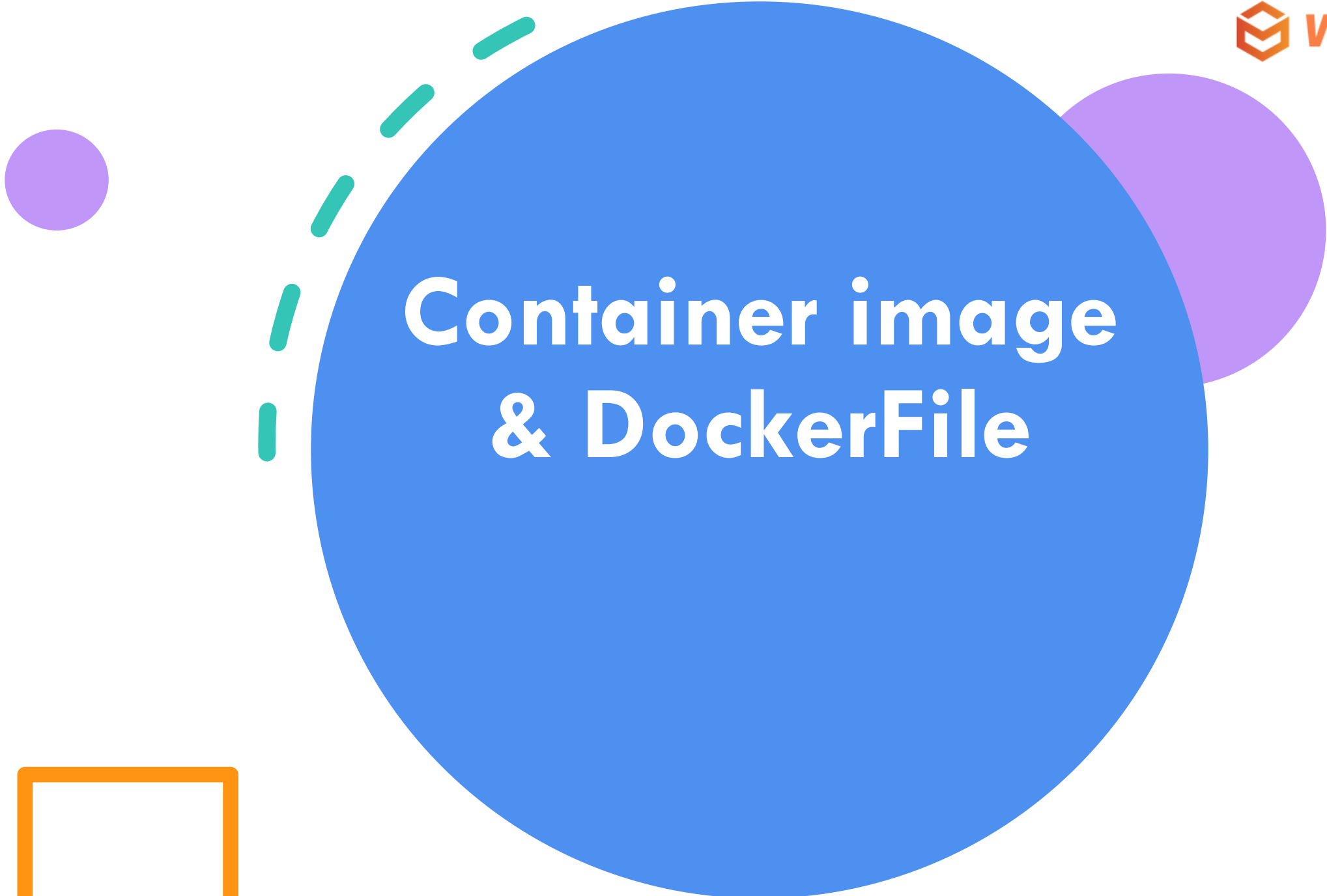
A large blue circle on the left side of the slide, with a smaller purple circle at its bottom-left edge.

Today's Lesson

Build Container image & Dockerfile

Image layer and Optimization

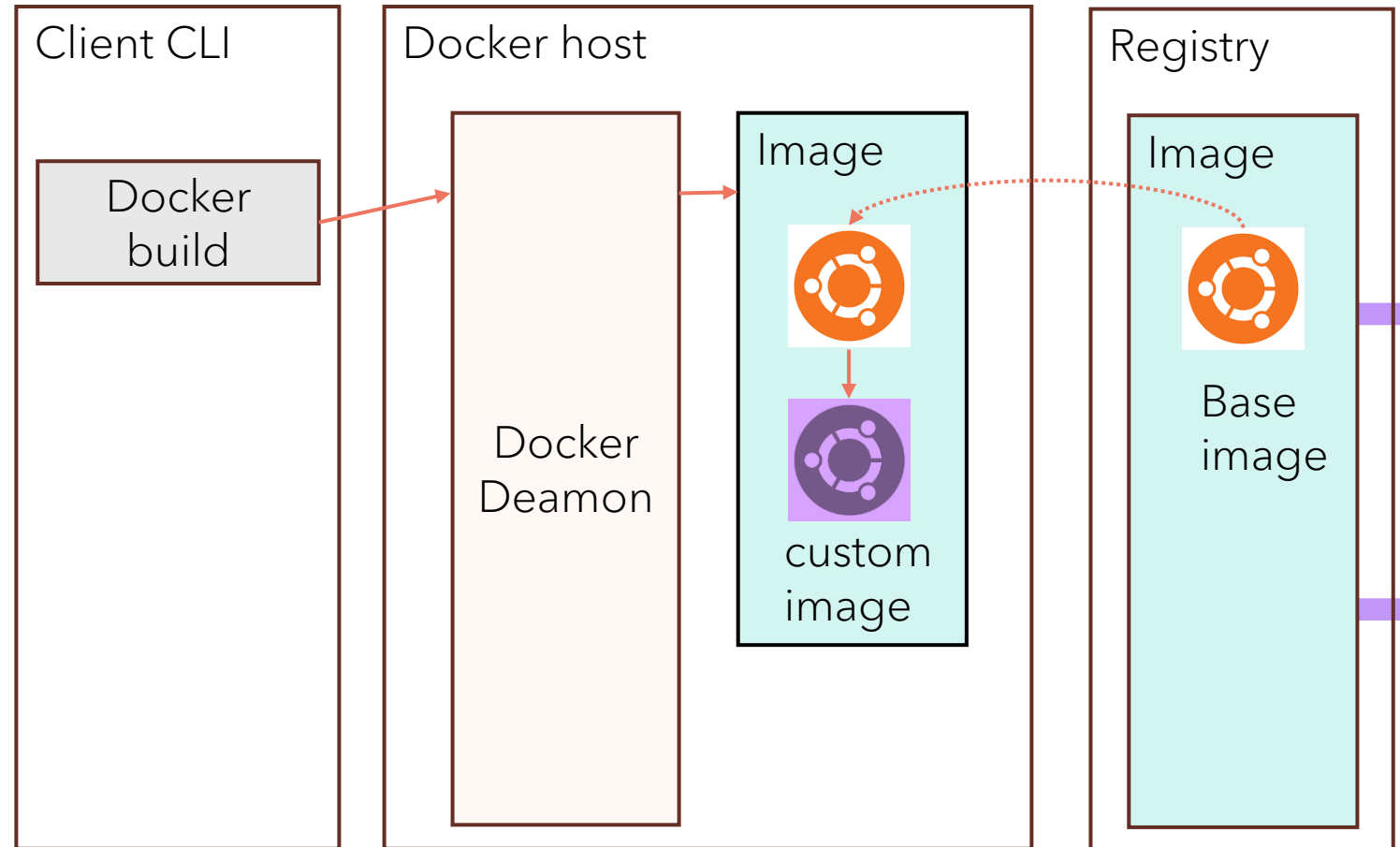
Docker compose



Container image & DockerFile

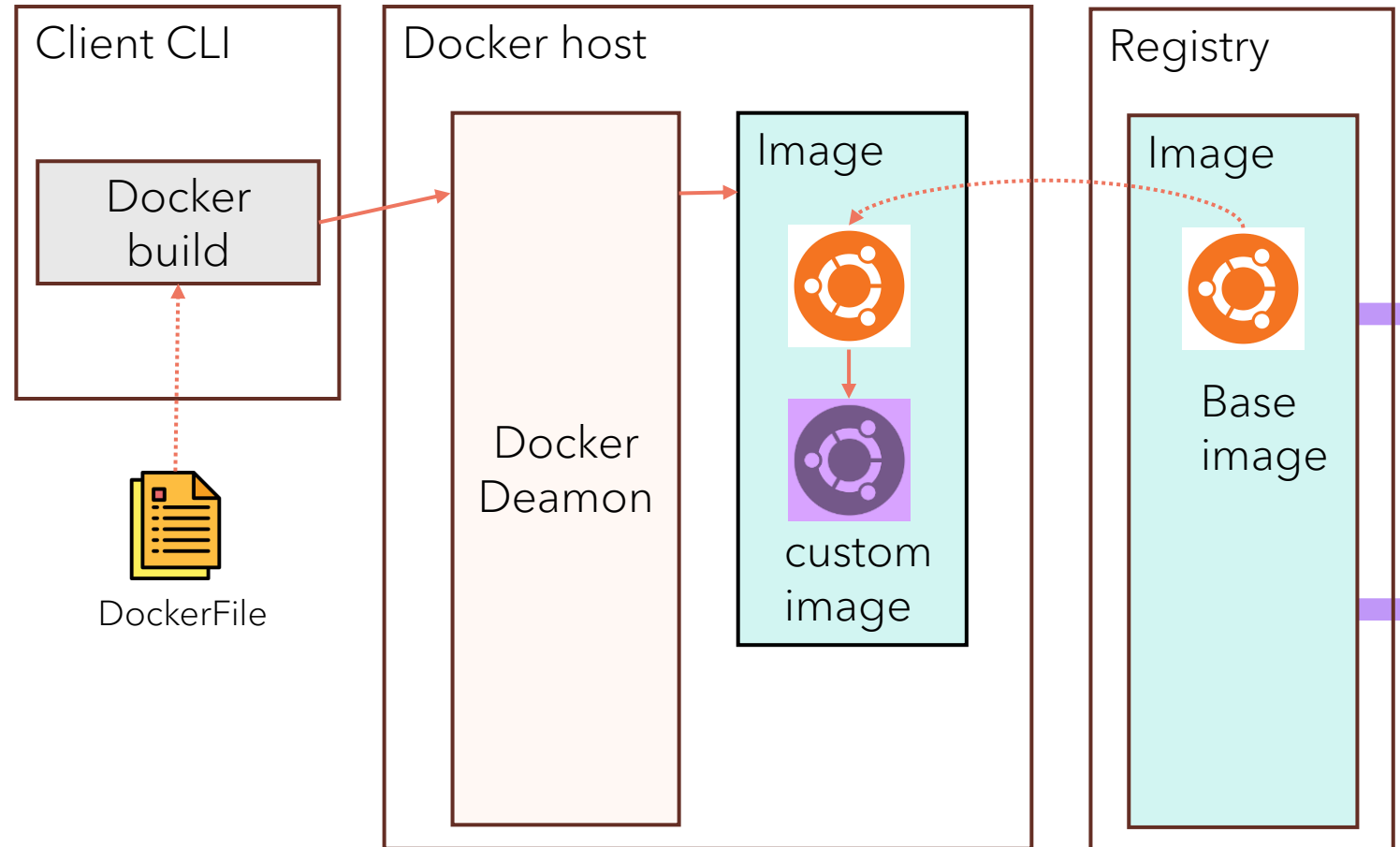
Dockerfile

- A Dockerfile is a **text file containing instructions** to build a Docker image. It automates the **creation of custom images** by specifying a base image, installing software, copying files, and configuring environment variables and network ports, ensuring an application can run consistently across different systems.



Dockerfile

- A **Dockerfile** is a **text file** containing instructions to build a Docker image. It automates the **creation of custom images** by specifying a base image, installing software, copying files, and configuring environment variables and network ports, ensuring an application can run consistently across different systems.



Dockerfile Structure



Instruction	Purpose
FROM	Sets base image (e.g., FROM ubuntu:22.04)
RUN	Executes commands in a new layer (e.g., install packages)
COPY / ADD	Copies files from host into the image
WORKDIR	Sets the working directory inside the container
EXPOSE	Declares the port the container listens on
CMD / ENTRYPOINT	Defines the default command to run on container start

```
# 1. Base image
FROM python:3.11-slim

# 2. Set working directory
WORKDIR /app

# 3. Copy files
COPY . .

# 4. Install dependencies
RUN pip install flask

# 5. Expose port
EXPOSE 5000

# 6. Run application
CMD ["python", "app.py"]
```

Image layer

- Each instruction in a Dockerfile creates a **new layer**.
- Layers are **cached** by Docker to speed up rebuilds.
- The order of commands affects performance and rebuild speed.



```
# 1. Base image
FROM python:3.11-slim

# 2. Set working directory
WORKDIR /app

# 3. Copy files
COPY . .

# 4. Install dependencies
RUN pip install flask

# 5. Expose port
EXPOSE 5000

# 6. Run application
CMD ["python", "app.py"]
```

Image layer



Instruction	Creates a new image layer?	Notes
FROM python:3.11-slim	✓ Yes	The base image is your first layer (it itself consists of multiple layers, but for your Dockerfile, it counts as one "stage" layer to start from).
WORKDIR /app	✓ Yes	Each metadata instruction like WORKDIR generates a new layer, though often quite small.
COPY ..	✓ Yes	Copies files into the image and creates a new layer containing them.
RUN pip install flask	✓ Yes	Executes a command during build, saves the result as a new layer.
EXPOSE 5000	✓ Yes (metadata layer)	Creates a new layer that only updates image metadata (port info).
CMD ["python", "app.py"]	✓ Yes (metadata layer)	Sets the default command – stored as image metadata.

1. Base image

FROM python:3.11-slim

2. Set working directory

WORKDIR /app

3. Copy files

COPY . .

4. Install dependencies

RUN pip install flask

5. Expose port

EXPOSE 5000

6. Run application

CMD ["python", "app.py"]

Image layer

- Which one is better? 1 or 2 ? Why?
- When Docker builds an image, each instruction creates a new layer. Layers that change less frequently should come **earlier** in the Dockerfile so Docker can reuse them from cache—this makes rebuilds faster.
- Your **first version is more efficient** as written, since dependencies will stay cached even if your code changes.

1

```
# 1. Base image
FROM python:3.11-slim

# 4. Install dependencies
RUN pip install flask

# 2. Set working directory
WORKDIR /app

# 3. Copy files
COPY . .

# 5. Expose port
EXPOSE 5000

# 6. Run application
CMD ["python", "app.py"]
```

2

```
# 1. Base image
FROM python:3.11-slim

# 2. Set working directory
WORKDIR /app

# 3. Copy files
COPY . .

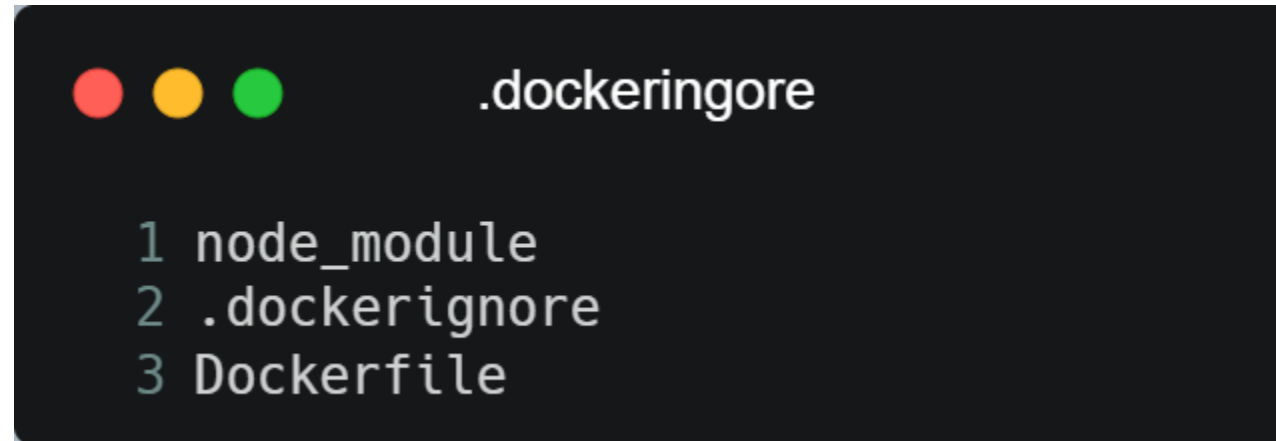
# 4. Install dependencies
RUN pip install flask

# 5. Expose port
EXPOSE 5000

# 6. Run application
CMD ["python", "app.py"]
```

.dockerignore

- **.dockerignore** tells Docker which files and folders to **exclude** from the build context when creating an image.
It helps make builds **faster**, images **smaller**, and prevents **sensitive or unnecessary files** from being copied into the container.



```
.dockerignore  
  
1 node_module  
2 .dockerignore  
3 Dockerfile
```

Multi Stage Build

- Docker Multi-stage builds are a feature in Docker that allows you to create more optimized and smaller Docker images by using multiple **FROM** instructions within a single Dockerfile. Each **FROM** instruction initiates a new "stage" in the build process.

This helps you:

- Keep final image size small
- Separate build-time dependencies (like compilers, libraries) from runtime
- Improve build efficiency and security

```
# Stage 1: Build environment
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .
```

```
# Stage 2: Runtime environment
FROM alpine:3.18
WORKDIR /app
COPY --from=builder /app/myapp .
CMD [ "./myapp" ]
```

Multi Stage Build

🌟 Benefits of multi stage build

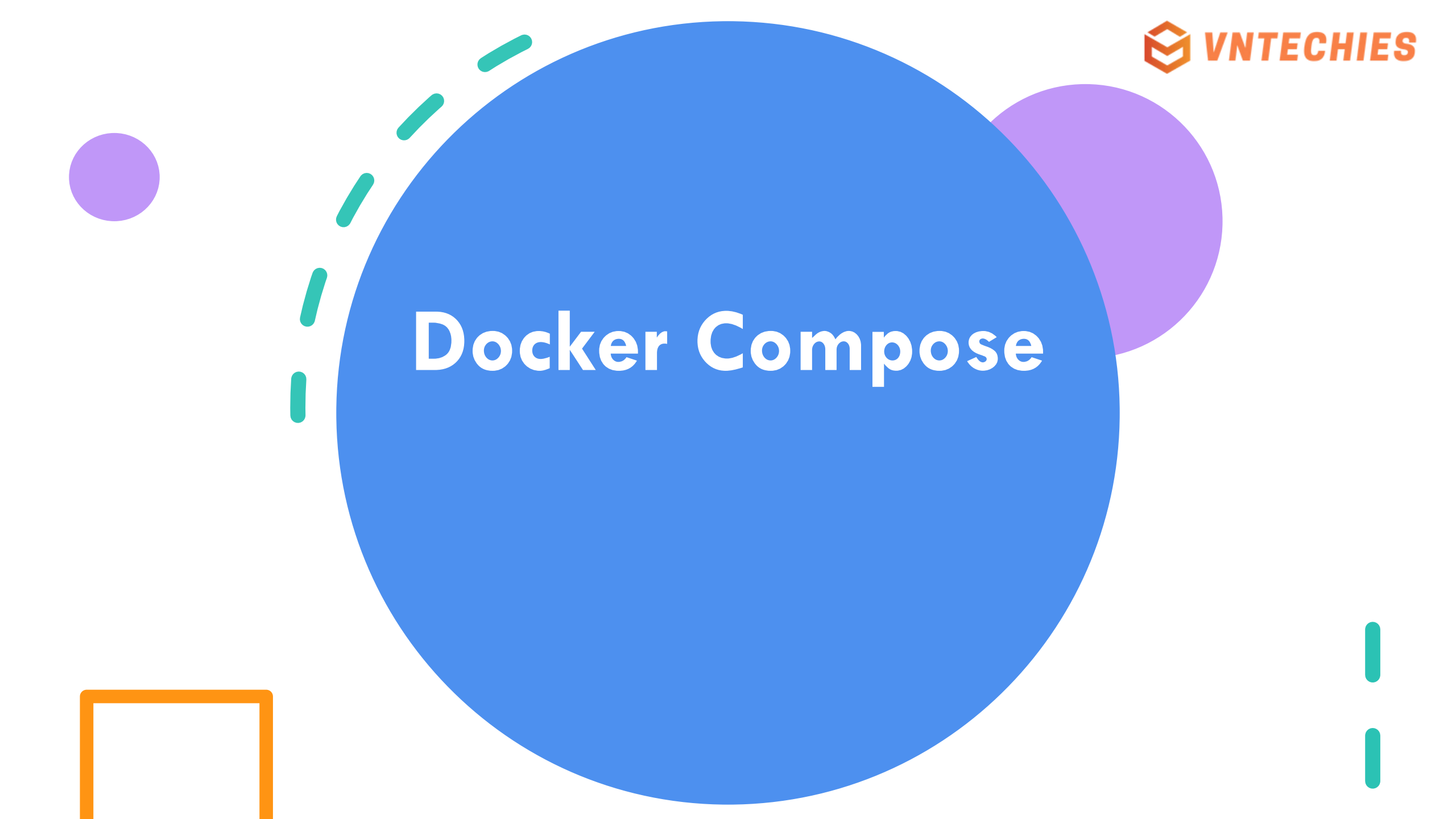
- ✓ **Smaller Image** - The final **alpine** image only contains the binary (a few MBs)
- ✓ **Security** - No compilers or build tools in production
- ✓ **Cleaner & Faster** - Each stage can be cached and reused
- ✓ **Reusable Builds** - You can have multiple targets in the same Dockerfile

We can have more than 2 stages, but typically we can categorize the stage like this

```
# Stage 1: Build environment
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .
```

```
# Stage 2: Runtime environment
FROM alpine:3.18
WORKDIR /app
COPY --from=builder /app/myapp .
CMD [ "./myapp" ]
```

Stage Type	Purpose
Builder	Compile or package artifacts
Runtime	Minimal environment to run the app



Docker Compose

Docker compose

Docker Compose is a tool that lets you define and manage multi-container Docker applications.

Instead of running multiple containers manually with multiple `docker run` commands, you can define them all in a single configuration file (`docker-compose.yml`).

Core Features

- Declarative configuration in YAML
- Single command to start/stop the whole app (`docker compose up/down`)
- Networking auto-created between your containers
- Named volumes for persistent data
- Build images automatically using Dockerfiles
- Environment variable support

```
version: "3.9"

services:
  app:
    build: .
    container_name: myapp
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASS=example
    depends_on:
      - db

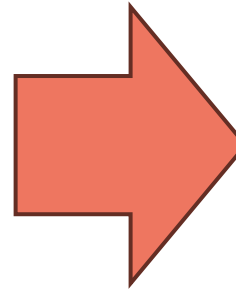
  db:
    image: mysql:8
    container_name: mydb
    environment:
      - MYSQL_ROOT_PASSWORD=example
      - MYSQL_DATABASE=myappdb
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Docker compose

```
.terminal

1 # Create network and volume
2 docker network create mynetwork
3 docker volume create db_data
4
5 # Run database
6 docker run -d \
7   --name mydb \
8   --network mynetwork \
9   -e MYSQL_ROOT_PASSWORD=example \
10  -e MYSQL_DATABASE=myappdb \
11  -v db_data:/var/lib/mysql \
12  mysql:8
13
14 # Build and run app
15 docker build -t myapp-image .
16 docker run -d \
17   --name myapp \
18   --network mynetwork \
19   -p 8080:8080 \
20   -e DB_HOST=db \
21   -e DB_USER=root \
22   -e DB_PASS=example \
23   myapp-image
```



```
version: "3.9"

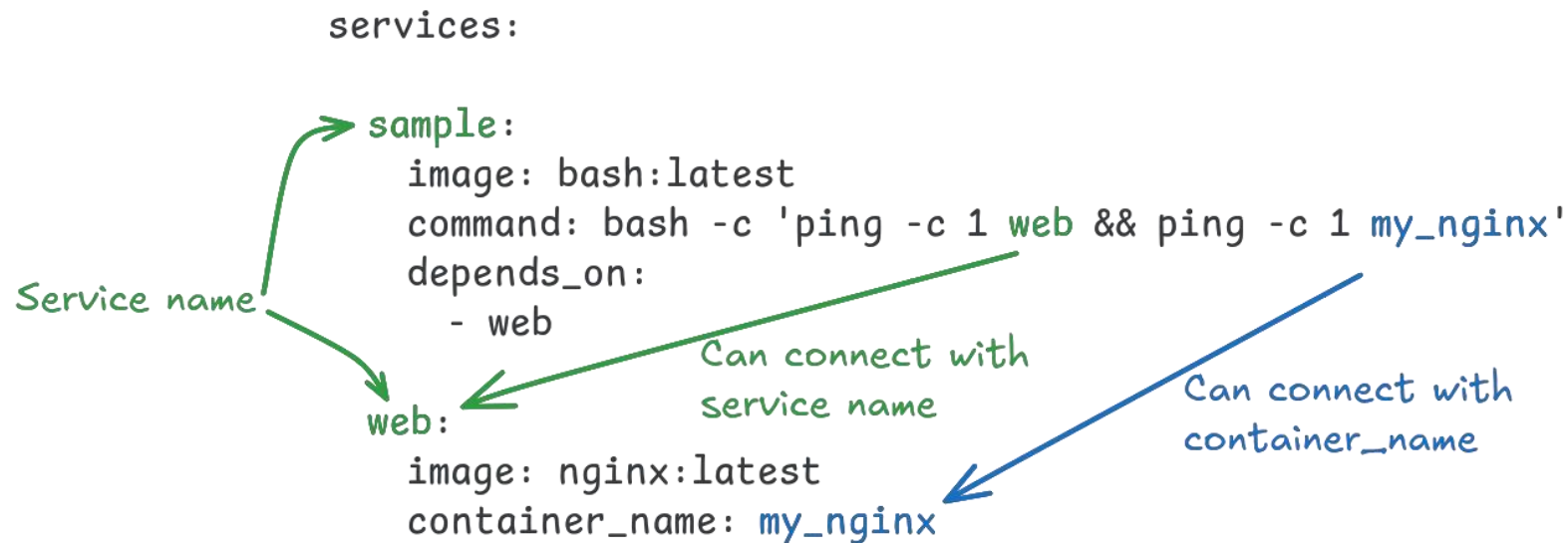
services:
  app:
    build: .
    container_name: myapp
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASS=example
    depends_on:
      - db

  db:
    image: mysql:8
    container_name: mydb
    environment:
      - MYSQL_ROOT_PASSWORD=example
      - MYSQL_DATABASE=myappdb
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Network and DNS resolution

- Service can call each other by service name or container name



Docker compose command

- Compose reference: <https://docs.docker.com/reference/compose-file/>
- Convert CLI to docker compose: <https://www.composerize.com/>

Command	Description
<code>docker compose up</code>	Build & start all services
<code>docker compose up -d</code>	Run in background
<code>docker compose down</code>	Stop and remove containers, networks
<code>docker compose ps</code>	List running services
<code>docker compose logs -f</code>	Follow logs
<code>docker compose build</code>	Rebuild images defined in Compose
<code>docker compose exec app sh</code>	Run a shell inside the <code>app</code> container



Practice Lab